

[Apache Spark](#)[Optimization](#)[Spark Sql](#)[Databricks](#)[Best Practices](#)

Join Strategies in Apache Spark — A hands-on approach!



Amar Kumar Gupta

Follow

9 min read · Aug 13, 2023

Open in app ↗

Sign up

Sign in

Medium



Search



Write



Basics and Pre-requisites:

a. What is a shuffle operation in Spark?

(Ans) Spark divides large datasets into smaller partitions that can be processed in parallel across multiple nodes in a cluster. A shuffle operation refers to the process of redistributing and reorganizing data across partitions in a distributed cluster.

Since it involves transferring data over the network, it is a costly operation and so unless necessary, it should be avoided.

b. What is a Hash Table?

(Ans) In the context of Apache Spark, a hash table is a data structure used to efficiently perform join operations between two or more datasets. *Hash*

Tables are created in-memory and hence attempting to create a hash table for large dataset may lead to an Out-of-Memory-Error.

Spark computes hash codes for the keys using a hash function and maps them to specific buckets in the hash table.

The hash function that is used to create the Hash Tables is deterministic (always gives the same output for a particular input).

Always remember that hash tables are always created on the smaller dataset.

Hash tables are efficient for join operations because they allow quick lookups based on the hash code of the join key, reducing the need to scan through the entire dataset.

c. Some configuration properties & commands to remember (related to join strategy):

i. `spark.sql.autoBroadcastJoinThreshold`: This determines the maximum size of the DataFrame that can be broadcasted. If the size of a DataFrame (in bytes) is less than or equal to the threshold, Spark will automatically choose a broadcast join strategy for that DataFrame during join operations.

```
# Set the autoBroadcastJoinThreshold property
# The value is specified in bytes, e.g., 10MB = 10 * 1024 * 1024 bytes
spark.conf.set("spark.sql.autoBroadcastJoinThreshold", 10 * 1024 * 1024)
```

() Setting `spark.sql.autoBroadcastJoinThreshold` to -1 and this will disable Broadcast Hash Join.*

ii. **spark.sql.join.preferSortMergeJoin**: This used to influence the query optimizer's choice of join strategy, specifically for choosing between the Sort-Merge Join and Hash Join algorithms. *By default, it is set to true* as this is preferred when datasets are big on both sides. Remember, performing data-sorting is also a costly operation in spark.

```
# Set the preferSortMergeJoin property
spark.conf.set("spark.sql.join.preferSortMergeJoin", "true") # or "false"
```

iii. **explain()**: You can find the type of join algorithm that Spark is using by calling *explain()* on the joined DataFrame. We can also use *_jdf.queryExecution().executedPlan()*. We'll see them practically later.

Join Strategies that are available in Apache Spark:

Apache Spark has the following five algorithms to choose from –

1. Broadcast Hash Join
2. Shuffle Hash Join
3. Shuffle Sort Merge Join
4. Broadcast Nested Loop Join
5. Cartesian Product Join (a.k.a Shuffle-and-Replicate Nested Loop Join)

(*) Let's discuss them one by one.

1. Broadcast Hash Join:

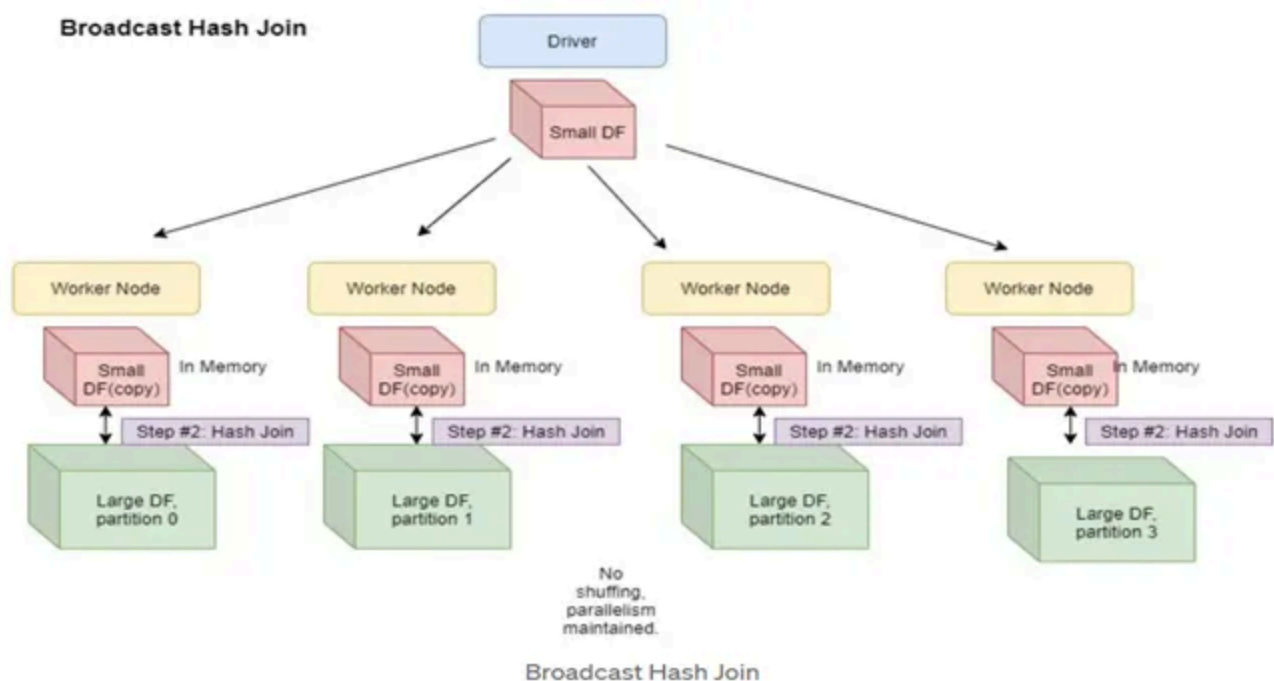
As the name suggests, this join-strategy involves two phases:

a. Broadcast Phase: The smaller dataset is broadcasted to every executor that has the partitioned big dataset.

b. Hash Phase: The smaller dataset is then hashed and bucketed at every executor. That is the hash table of the small dataset that has the info regarding hash-key, join-key, bucket#,etc. is present at every executor.

(*) Then in each partition (executor), the records from the bigger dataset only to those buckets where there's a possibility of a match, and if the join-key matches the records are joined together.

(*) This neither includes a shuffle nor a sort-operation and hence it is the fastest join strategy in most of the scenarios.



Practical:

We have two DataFrames df1 and df2 with one column in each — id1 and id2 respectively. We are doing a simple join on id1 and id2. (For practicals we'll be using Pyspark)

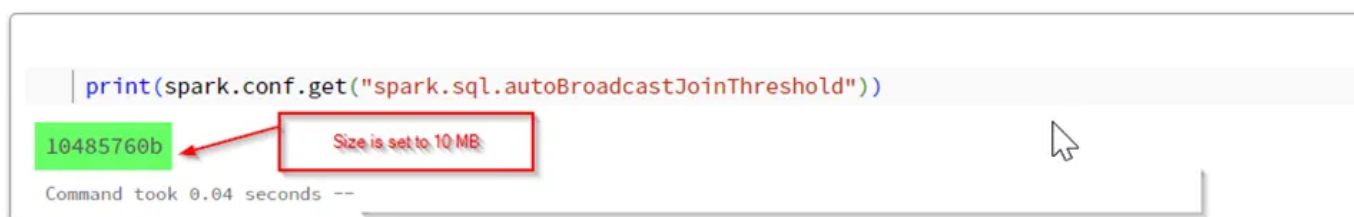
Let's create those two dataframes — df1 & df2.

```
# Creating a pyspark df from a list of integer values|
integer_values = [10, 20, 20, 30, 40, 10, 40, 20, 20, 20, 20, 50]
df1 = spark.createDataFrame([(value,) for value in integer_values], ["id1"])
```

```
integer_values = [30, 20, 40, 50]
df2 = spark.createDataFrame([(value,) for value in integer_values], ["id2"])
```

Let's see the value of autoBroadcastJoinThreshold:

```
| print(spark.conf.get("spark.sql.autoBroadcastJoinThreshold"))
```



10485760b

Size is set to 10 MB

Command took 0.04 seconds --

Let's perform the join operation:

Here we have used the “broadcast” hint to specify that df2 needs to be broadcasted.

```
from pyspark.sql.functions import broadcast
joined_df = df1.join(broadcast(df2), df1.id1 == df2.id2)
joined_df.display()
```

► (3) Spark Jobs

►  joined_df: pyspark.sql.dataframe.DataFrame = [id1: long, id2: long]

Table  +

	id1	id2
4	40	40
5	40	40
6	20	20
7	20	20
8	20	20
9	20	20
10	50	50

 10 rows | 0.49 seconds runtime

Let's examine the query-plan that was executed by spark optimizer:

```
execution_plan = joined_df._jdf.queryExecution().executedPlan()
```

```
# Show the human-readable execution plan
```

```
print(execution_plan.toString())
```

_jdf is used to convert pyspark df to a java df in order to access the functions specific to java

```
AdaptiveSparkPlan isFinalPlan=false
```

```
+-- BroadcastHashJoin [id1#12L], [id2#17L], Inner, BuildRight, false
  :- Filter isNotNull(id1#12L)
  : +- Scan ExistingRDD[id1#12L]
  +- Exchange SinglePartition, EXECUTOR_BROADCAST, [plan_id=701]
    +- Filter isNotNull(id2#17L)
      +- Scan ExistingRDD[id2#17L]
```

(*) Query plan is always read from “Bottom-to-Up” approach.

(*) You can clearly see “**BroadcastHashJoin**” was used to join df1 and df2.

Some key points regarding Broadcast Hash Join:

- a. Only supported for equi-joins (joins with ‘=’ condition).
- b. Supports all types of joins (inner, left, right) except full-outer-join.
- c. If the DataFrame that is being broadcasted is too large, it can lead to excessive memory consumption and potential out-of-memory errors.
- d. Broadcasting the smaller DataFrame to all worker nodes can generate significant network traffic, especially when dealing with *a large number of nodes* and can be very costly.

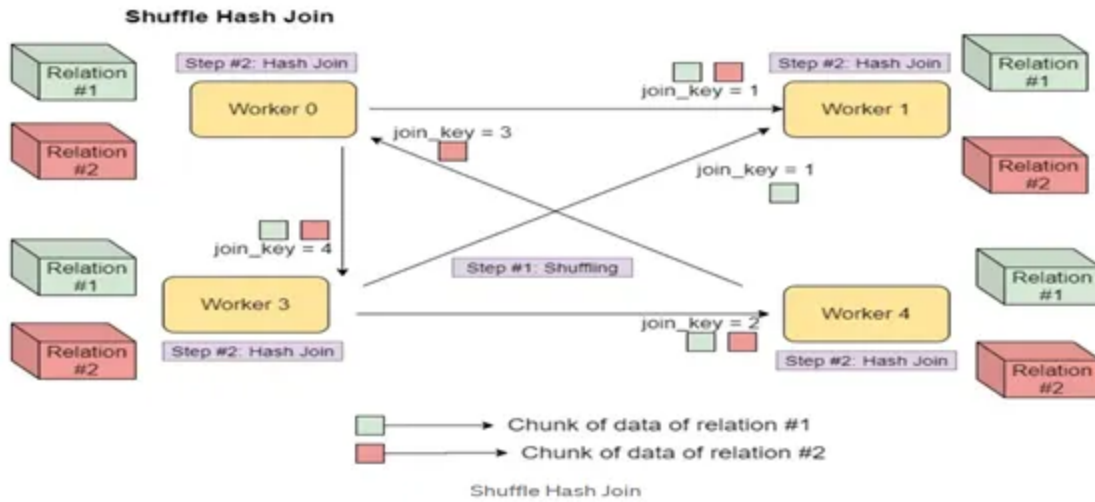
2. Shuffle Hash Join:

As the name suggests, involves two phases:

- a. **Shuffle phase** — Both datasets are shuffled. So, the same keys from both sides end up in the same partition or task.
- b. **Hash Join phase** — Smaller side data is hashed and bucketted and hash joined with the bigger side in all the partitions.

(*) *Shuffle Hash Join is different from Broadcast Hash Join* because the entire dataset is not broadcasted instead both datasets are shuffled and then the smallest side data is hashed and bucketted and hash joined with the bigger side in all the partitions.

(*) Sorting is not needed with Shuffle Hash Joins inside the partitions.



Practical:

Considering the same data-frames — df1 and df2, let's try to do a Shuffle Hash Join.

Get Amar Kumar Gupta's stories in your inbox

Join Medium for free to get updates from this writer.

Enter your email

Subscribe

☐ Remember me for faster sign in

Step1: Let's disable sort-merge-join that is active by default and set *autoBroadcastJoinThreshold* to smaller value. So, Spark can choose to use Shuffle Hash Join over Sort Merge Join.

```
spark.conf.set("spark.sql.join.preferSortMergeJoin", "false")
```

```
spark.conf.set("spark.sql.autoBroadcastJoinThreshold", "2")
```

Let's verify its value again:

```
| print(spark.conf.get("spark.sql.join.preferSortMergeJoin"))
```

false

```
| print(spark.conf.get("spark.sql.autoBroadcastJoinThreshold"))
```

2

Step2: Let's perform a Shuffle Hash Join using hints:

```
from pyspark.sql.functions import broadcast
joined_df = df1.join(df2.hint("SHUFFLE_HASH"), df1.id1 == df2.id2)
joined_df.display()
```

► (3) Spark Jobs

►  joined_df: pyspark.sql.dataframe.DataFrame = [id1: long, id2: long]

Table ▼ +

	id1	id2
4	40	40
5	20	20
6	20	20
7	40	40
8	50	50
9	20	20
10	20	20

↓ 10 rows | 1.09 seconds runtime

Step3: Let's see the executed query plan:

```
execution_plan = joined_df._jdf.queryExecution().executedPlan()
```

```
# Show the human-readable execution plan
print(execution_plan.toString())
```

```
AdaptiveSparkPlan isFinalPlan=true
+- == Final Plan ==
  *(3) ShuffledHashJoin [id1#2L], [id2#7L], Inner, BuildRight
    :- AQEShuffleRead coalesced
    : +- ShuffleQueryStage 0, Statistics(sizeInBytes=192.0 B, rowCount=12, isRuntime=true)
    :   +- Exchange hashpartitioning(id1#2L, 200), ENSURE_REQUIREMENTS, [plan_id=441]
    :     +- *(1) Filter isnotnull(id1#2L)
    :       +- *(1) Scan ExistingRDD[id1#2L]
    +- AQEShuffleRead coalesced
      +- ShuffleQueryStage 1, Statistics(sizeInBytes=64.0 B, rowCount=4, isRuntime=true)
        +- Exchange hashpartitioning(id2#7L, 200), ENSURE_REQUIREMENTS, [plan_id=445]
          +- *(2) Filter isnotnull(id2#7L)
            +- *(2) Scan ExistingRDD[id2#7L]
```

Some key points regarding Shuffle Hash Join:

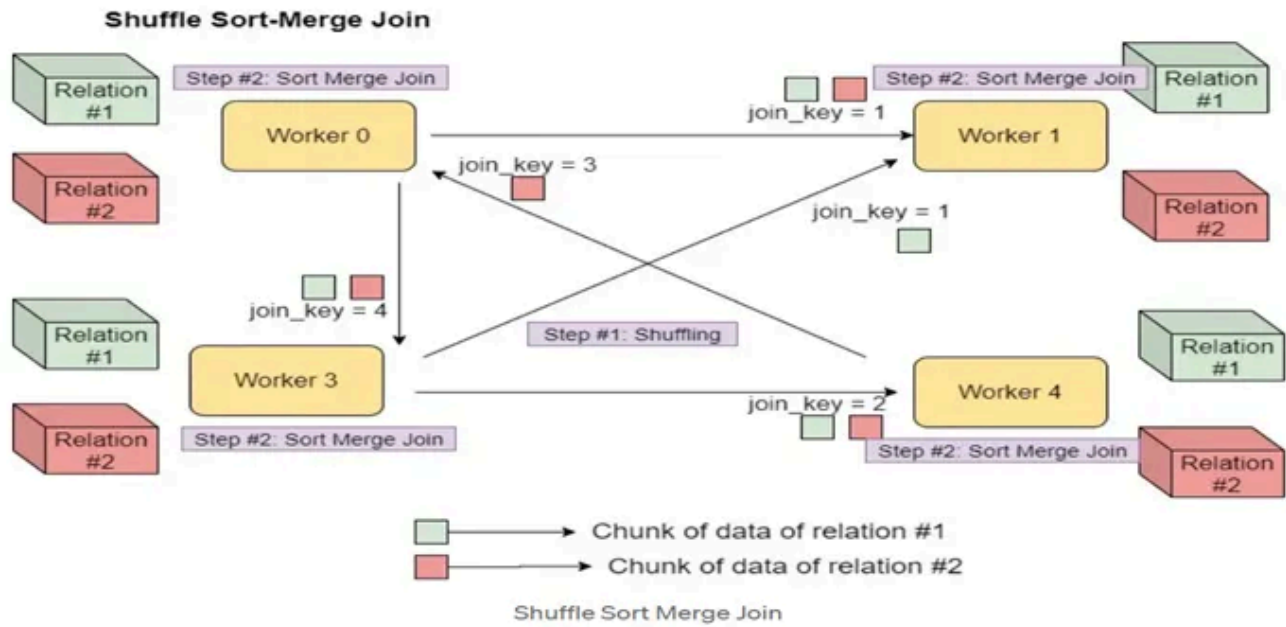
- a. Works only for equi-joins.
- b. Works for all join types.
- c. Works well when a dataset cannot be broadcasted but one side of partitioned data after shuffling will be small enough for hash join.
- d. Faster than a sort merge join since sorting is not involved.
- e. Works for a balanced dataset but not for a skewed dataset as it may lead to out-of-memory errors.

3. Shuffler Sort Merge Join:

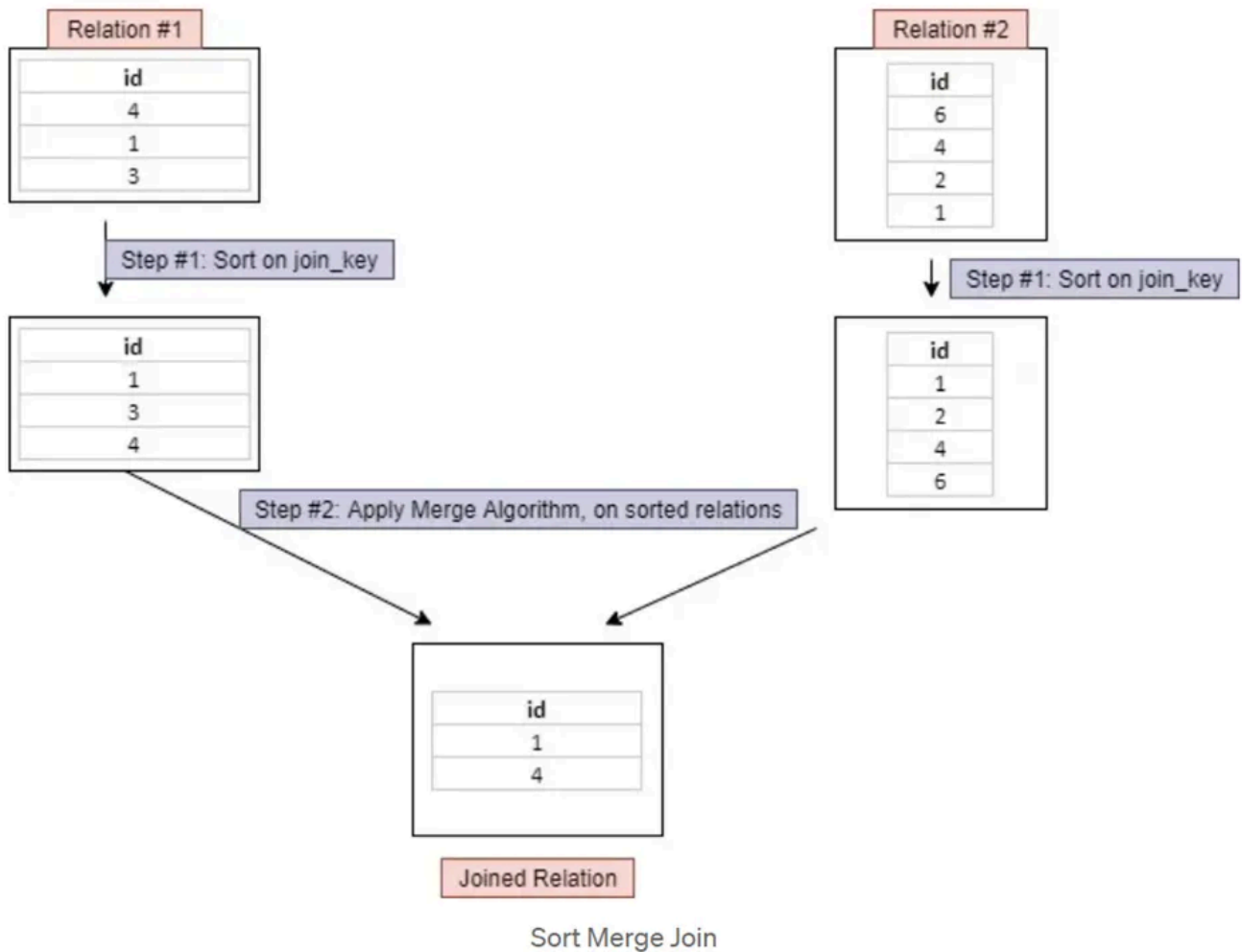
As the name indicates, it has 3 phases.

- a. **Shuffle Phase:** both datasets are shuffled.
- b. **Sort Phase:** records are sorted by key on both sides.
- c. **Merge Phase:** iterate over both sides and join based on the join key.

(*) Shuffle Sort Merge Join is preferred when both datasets are big and cannot fit in memory — with or without shuffle.



(*) Sort-Merge Step:



(*) **Reason to sort the keys before merge (join) operation:** A join is performed by iterating over the records on the sorted dataset. Since the dataset is sorted the merge or the join operation is stopped for an element as soon as a key mismatch is encountered. So a join attempt is not performed on all keys.

For example, when id = 1 from the 1st dataset is attempted to match, it must iterate through on the other side (2nd dataset) only till it encounters '2' in the second dataset. As the records are sorted there is no need to iterate through the entire records on the other side (as after '2' there's no possibility to find another id='1' in the second dataset).

Practical:

Step1: Set the below configuration properties to force a Sort Merge join.

```
spark.conf.set("spark.sql.join.preferSortMergeJoin", "true")
spark.conf.set("spark.sql.autoBroadcastJoinThreshold", "-1")
```

Step2: Perform a shuffle-sort-merge-join by specifying appropriate hints.

```
from pyspark.sql.functions import broadcast
joined_df = df1.join(df2.hint("MERGE"), df1.id1 == df2.id2)
joined_df.display()
```

► (3) Spark Jobs

►  joined_df: pyspark.sql.dataframe.DataFrame = [id1: long, id2: long]

Table ▾ +

	id1 ▲	id2 ▲	
1	20	20	
2	20	20	
3	20	20	
4	20	20	
5	20	20	
6	20	20	
7	30	30	

↓ 10 rows | 1.04 seconds runtime

Step3: Let's see the executed query plan.

```
joined_df.explain()
```

```
== Physical Plan ==
AdaptiveSparkPlan isFinalPlan=false
+- SortMergeJoin [id1#2L], [id2#7L], Inner
   :- Sort [id1#2L ASC NULLS FIRST], false, 0
      :  +- Exchange hashpartitioning(id1#2L, 200), ENSURE_REQUIREMENTS, [plan_id=626]
      :    +- Filter isnotnull(id1#2L)
      :      +- Scan ExistingRDD[id1#2L]
   +- Sort [id2#7L ASC NULLS FIRST], false, 0
      +- Exchange hashpartitioning(id2#7L, 200), ENSURE_REQUIREMENTS, [plan_id=627]
         +- Filter isnotnull(id2#7L)
            +- Scan ExistingRDD[id2#7L]
```

Some Important Points regarding Shuffle-Sort-Merge Join:

- a. Works only on equi-joins.
- b. Works on all join-types.
- c. Both shuffle and sort are expensive operations. Use this join only when Broadcast Hash Join and Shuffle Hash Join are not possible.

4. Broadcast Nested Loop Join:

Broadcast Nested Loop join works by broadcasting one of the entire datasets and performing a nested loop to join the data. So essentially every record from dataset 1 is attempted to join with every record from dataset 2.

Practical:

We don't change the default values for both *spark.sql.join.preferSortMergeJoin* & *spark.sql.autoBroadcastJoinThreshold*.

```
print(spark.conf.get("spark.sql.join.preferSortMergeJoin"))
```

true

```
print(spark.conf.get("spark.sql.autoBroadcastJoinThreshold"))
```

10485760b

Let's perform the join:

```
from pyspark.sql.functions import broadcast
joined_df = df1.join(broadcast(df2), df1.id1 >= df2.id2)
joined_df.display()
```

► (3) Spark Jobs

►  joined_df: pyspark.sql.dataframe.DataFrame = [id1: long, id2: long]

Table  

	id1	id2
1	20	20
2	20	20
3	30	30
4	30	20
5	40	30
6	40	20
7	40	40

 18 rows | 0.65 seconds runtime

Let's see the executed query plan:

```
joined_df.explain()
```

```
== Physical Plan ==
AdaptiveSparkPlan isFinalPlan=false
+- BroadcastNestedLoopJoin BuildRight, Inner, (id1#2L >= id2#7L)
   :- Filter isnotnull(id1#2L)
   :   +- Scan ExistingRDD[id1#2L]
   +- Exchange SinglePartition, EXECUTOR_BROADCAST, [plan_id=940]
      +- Filter isnotnull(id2#7L)
         +- Scan ExistingRDD[id2#7L]
```

Some important points regarding Broadcast Nested Loop Join:

- a. Works for both equi and non-equi joins.
- b. Works for all join types.
- c. When you specify broadcast as one of the hints and it's a non-equi join, then spark query optimizer always chooses this strategy.
- d. This join will not work when both sides “are big enough for broadcasting and you could see Out Of Memory exceptions.
- e. It is quite slow because essentially every record from dataset 1 is attempted to join with every record from dataset 2.

5. Cartesian Product Join:

Cartesian Product Join (a.k.a. Shuffle-and-Replication Nested Loop) join works very similar to a Broadcast Nested Loop join except the dataset is not broadcasted.

Shuffle-and-Replication does not mean a “true” shuffle as in records with the same keys are sent to the same partition. Instead, the entire partition of the dataset is sent over or replicated to all the partitions for a full cross or nested loop join.

Practical:

Let's perform the join:

```
from pyspark.sql.functions import broadcast
joined_df = df1.join(df2.hint("SHUFFLE_REPLICATE_NL"), df1.id1 >= df2.id2)
joined_df.display()
```

▶ (3) Spark Jobs

▶ joined_df: pyspark.sql.dataframe.DataFrame = [id1: long, id2: long]

Table ▾ +			
	id1	id2	
1	20	20	
2	20	20	
3	30	30	
4	40	30	
5	30	20	
6	40	20	
7	40	40	
↓ 18 rows 0.80 seconds runtime			

Let's see the executed query plan:

```
joined_df.explain()
```

```
== Physical Plan ==
```

```
CartesianProduct (id1#2L >= id2#7L)
```

```
:- *(1) Filter isnotnull(id1#2L)
```

```
: +- *(1) Scan ExistingRDD[id1#2L]
```

```
+ - *(2) Filter isnotnull(id2#7L)
```

```
    +- *(2) Scan ExistingRDD[id2#7L]
```

Some important points regarding Cartesian Product Join:

- a. Works for both equi and non-equi joins.
- b. Works only for inner like joins.
- c. Very expensive join algorithm.
- d. High possibility of Out-of-Memory Exceptions.
- e. For non-equi join, if “*SHUFFLE_REPLICATE_NL*” hint is provided, then query optimizer chooses this join strategy.

Conclusion:

In conclusion, the selection of the right join strategy in Apache Spark is a critical decision that can significantly impact the performance and scalability of your data processing tasks. It's essential to consider factors such as data size, distribution, available memory, and cluster resources when choosing a join strategy. By making informed decisions, you can harness the power of Spark's optimization techniques to maximize the efficiency of your data operations. As you navigate the world of distributed data processing, a deep understanding of join strategies empowers you to unlock the true potential of Apache Spark.

Apache Spark

+

Optimization

+

Spark Sql

+

Databricks

+

Best Practices

+



Written by Amar Kumar Gupta

197 followers · 5 following

Follow

Fresher in learning.
